CredShields

Smart Contract Audit

May 7, 2026 • CONFIDENTIAL

Description

This document details the process and result of the Smart Contract audit performed by CredShields Technologies PTE. LTD. on behalf of Octos between April 30, 2026, and May 4, 2026. A retest was performed on May 7, 2026.

Author

Shashank (Co-founder, CredShields) shashank@CredShields.com

Reviewers

Aditya Dixit (Research Team Lead), Shreyas Koli (Auditor), Sanket Salavi (Auditor), Prasad Kuri (Auditor), Neel Shah (Auditor)

Prepared for

Octos

Table of Contents

1. Executive Summary ----------

Octos engaged CredShields to perform a smart contract audit from April 30, 2026, to May 4, 2026. During this timeframe, 6 vulnerabilities were identified. **A retest was performed on May 7, 2026, and all the bugs have been addressed.**

During the audit, 1 vulnerability was found with a severity rating of either High or Critical. These vulnerabilities represent the greatest immediate risk to "Octos" and should be prioritized for remediation.

The table below shows the in-scope assets and a breakdown of findings by severity per asset. Section 2.3 contains more information on how severity is calculated.

Assets in Scope	Critical	High	Medium	Low	info	Gas	Σ
Octos Betting Contracts	0	1	1	2	2	0	**6**

Table: Vulnerabilities Per Asset in Scope

The CredShields team conducted the security audit to focus on identifying vulnerabilities in Octos Betting Contract's scope during the testing window while abiding by the policies set forth by Octos's team.

State of Security

To maintain a robust security posture, it is essential to continuously review and improve upon current security processes. Utilizing CredShields' continuous audit feature allows both Octos's internal security and development teams to not only identify specific vulnerabilities but also gain a deeper understanding of the current security threat landscape.

To ensure that vulnerabilities are not introduced when new features are added, or code is refactored, we recommend conducting regular security assessments. Additionally, by analyzing the root cause of resolved vulnerabilities, the internal teams at Octos can implement both manual and automated procedures to eliminate entire classes of vulnerabilities in the future. By taking a proactive approach, Octos can future-proof its security posture and protect its assets.

2. The Methodology ----------

Octos engaged CredShields to perform a Octos Betting Contract audit. The following sections cover how the engagement was put together and executed.

2.1 Preparation Phase

The CredShields team meticulously reviewed all provided documents and comments in the smart contract code to gain a thorough understanding of the contract's features and functionalities. They meticulously examined all functions and created a mind map to systematically identify potential security vulnerabilities, prioritizing those that were more critical and business-sensitive for the refactored code. To confirm their findings, the team deployed a self-hosted version of the smart contract and performed verifications and validations during the audit phase.

A testing window from April 30, 2026, to May 4, 2026, was agreed upon during the preparation phase.

2.1.1 Scope

During the preparation phase, the following scope for the engagement was agreed upon:

IN SCOPE ASSETS
https://github.com/Octosmeme/octos_space_ladder/tree/72f95ff0c07e3b0a057542424ad89d84f497855c

2.1.2 Documentation

Documentation was not required as the code was self-sufficient for understanding the project.

2.1.3 Audit Goals

CredShields employs a combination of in-house tools and thorough manual review processes to deliver comprehensive smart contract security audits. The majority of the audit involves manual inspection of the contract's source code, guided by OWASP's Smart Contract Security Weakness Enumeration (SCWE) framework and an extended, self-developed checklist built from industry best practices. The team focuses on deeply understanding the contract's core logic, designing targeted test cases, and assessing business logic for potential vulnerabilities across OWASP's identified weakness classes.

CredShields aligns its auditing methodology with the OWASP Smart Contract Security projects, including the Smart Contract Security Verification Standard (SCSVS), the Smart Contract Weakness Enumeration (SCWE), and the Smart Contract Secure Testing Guide (SCSTG). These frameworks, actively contributed to and co-developed by the CredShields team, aim to bring consistency, clarity, and depth to smart contract security assessments. By adhering to these OWASP standards, we ensure that each audit is performed against a transparent, community-driven, and technically robust baseline. This approach enables us to deliver structured, high-quality audits that address both common and complex smart contract vulnerabilities systematically.

2.2 Retesting Phase

Octos is actively partnering with CredShields to validate the remediations implemented towards the discovered vulnerabilities.

2.3 Vulnerability classification and severity

CredShields follows OWASP's Risk Rating Methodology to determine the risk associated with discovered vulnerabilities. This approach considers two factors - Likelihood and Impact - which are evaluated with three possible values - **Low**, **Medium**, and **High**, based on factors such as Threat

agents, Vulnerability factors, and Technical and Business Impacts. The overall severity of the risk is calculated by combining the likelihood and impact estimates.

Overall Risk Severity				
Impact	HIGH	Medium	High	Critical
	MEDIUM	Low	Medium	High
	LOW	None	Low	Medium
		LOW	MEDIUM	HIGH
Likelihood				

Overall, the categories can be defined as described below -

1. **Informational**

   We prioritize technical excellence and pay attention to detail in our coding practices. Our guidelines, standards, and best practices help ensure software stability and reliability. Informational vulnerabilities are opportunities for improvement and do not pose a direct risk to the contract. Code maintainers should use their own judgment on whether to address them.

2. **Low**

   Low-risk vulnerabilities are those that either have a small impact or can't be exploited repeatedly or those the client considers insignificant based on their specific business circumstances.

3. **Medium**

   Medium-severity vulnerabilities are those caused by weak or flawed logic in the code and can lead to exfiltration or modification of private user information. These vulnerabilities

can harm the client's reputation under certain conditions and should be fixed within a specified timeframe.

4. **High**

   High-severity vulnerabilities pose a significant risk to the Smart Contract and the organization. They can result in the loss of funds for some users, may or may not require specific conditions, and are more complex to exploit. These vulnerabilities can harm the client's reputation and should be fixed immediately.

5. **Critical**

   Critical issues are directly exploitable bugs or security vulnerabilities that do not require specific conditions. They often result in the loss of funds and Ether from Smart Contracts or users and put sensitive user information at risk of compromise or modification. The client's reputation and financial stability will be severely impacted if these issues are not addressed immediately.

6. **Gas**

   To address the risk and volatility of smart contracts and the use of gas as a method of payment, CredShields has introduced a "Gas" severity category. This category deals with optimizing code and refactoring to conserve gas.

2.4 CredShields staff

The following individual at CredShields managed this engagement and produced this report:

- Shashank, Co-founder CredShields shashank@CredShields.com

Please feel free to contact this individual with any questions or concerns you have about the engagement or this document.

3. Findings Summary ----------

This chapter contains the results of the security assessment. Findings are sorted by their severity and grouped by asset and OWASP SCWE classification. Each asset section includes a summary highlighting the key risks and observations. The table in the executive summary presents the total number of identified security vulnerabilities per asset, categorized by risk severity based on the OWASP Smart Contract Security Weakness Enumeration framework.

3.1 Findings Overview

3.1.1 Vulnerability Summary

During the security assessment, 6 security vulnerabilities were identified in the asset.

Vulnerability Title	Severity	Vulnerability Type	Status
Multiplier above 2x enables risk-free guaranteed profit via dual-side hedging	High	Business Logic (SC03-LogicErrors)	Fixed
fulfillRandomWords Reverts Allow Owner To Re-Roll VRF Randomness	Medium	Randomness Manipulation / VRF Integration Flaw	Fixed
Owner can selectively void unfavorable rounds after observing the committed VRF random word	Low	Business Logic (SC03-LogicErrors)	Fixed
Sybil Attacker Can Brick Resolution By Inflating Player Arrays	Low	Denial of Service (SCWE-087)	Fixed
Owner and pool-wallet operator hold unilateral authority over user funds and protocol liveness	Informational	Centralization Risk	Fixed
Fee-on-transfer and rebasing tokens are silently unsupported, causing accounting drift and pool-wallet over payouts	Informational	Token-compatibility	Fixed

Table: Findings and Remediations

5. Bug Reports ----------

Bug ID #H001 [Fixed]

Multiplier above 2x enables risk-free guaranteed profit via dual-side hedging

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
High

Description
The contract pays winners at a configurable multiplier multiplierBps constrained to the inclusive range 1.5x to 3.0x, while each round resolves to one of two binary outcomes (ODD or EVEN) with probability exactly one half each, with no fee, rake, or house edge applied to payouts. Because placeBet maintains oddStakeOf and evenStakeOf as two independent per-side balances and never checks that a player has bet on the opposite side, the same address is free to stake on both sides simultaneously within a single round.

An attacker who stakes S on ODD and S on EVEN spends 2S, is guaranteed to win exactly one side regardless of randomness, receives S x multiplierBps / 10_000 from the winning side, forfeits S on the losing side, and nets S x (multiplierBps / 10_000 – 2). Whenever multiplierBps > 20_000 this is strictly positive, every round produces a guaranteed profit independent of the random outcome, the player's role, or any external condition. The breakeven multiplier for any fair binary game without a fee is exactly 2.0x, and the protocol's MAX_MULTIPLIER_BPS = 30_000 constant permits configurations up to 50% above that breakeven point.

Affected Code

- SpaceLadderBettingV7.sol#L34

Impacts
The pool wallet loses approximately S x (M – 2) per attacker per round, equating to a 50% risk-free return at the maximum 3.0x setting and 25% at a mid-range 2.5x setting; with the documented 288-rounds-per-day cadence an attacker can compound nearly 288 guaranteed wins daily until poolWallet is drained, after which honest single-side winners will see their withdrawWinnings calls revert with InsufficientPool on a first-come-first-served basis.

Remediation
It is suggested to lower MAX_MULTIPLIER_BPS to a value below 20_000.

Retest
This issue has been fixed by setting MAX_MULTIPLIER_BPS to 19_900.

Bug ID #M001 [Fixed]

fulfillRandomWords Reverts Allow Owner To Re-Roll VRF Randomness

Vulnerability Type
Randomness Manipulation / VRF Integration Flaw

Severity
Medium

Description
The contract integrates Chainlink VRF to source randomness for round resolution, with fulfillRandomWords() acting as the coordinator's callback entry point. Per Chainlink's official VRF integration guide, this callback must never revert, if it does, the coordinator silently drops the result and never retries.

The implementation in SpaceLadderBettingV7.sol contains three revert statements across fulfillRandomWords() and the inner _commitRandomWord(). Two of them (UnknownVrfRequest and NotDrawing) trigger whenever round state is reset before the callback lands, which is exactly the state produced by emergencyResolve() and resolveExternal().

Because EMERGENCY_RESOLVE_DELAY is only 2 minutes, the owner can race a pending VRF callback: invoke emergencyResolve() to reset vrfRequestId and isDrawing, force the inbound callback to revert, and then issue a fresh VRF request for a new random word. The on-chain re-derivation of side and legs from currentRandomWord is meaningless once the input itself can be discarded and resampled.

Affected Code

- SpaceLadderBettingV7.sol#L288-L365

Impacts
The reverts violate Chainlink's VRF contract, any callback arriving after a state reset is silently dropped, and the round's randomness is permanently lost. The 2-minute emergency-resolve window converts this into an owner-controlled re-roll primitive: the owner can discard any unfavorable VRF result and request fresh randomness at the cost of one VRF fee and ~10 minutes per attempt.

Since the outcome is derived from only two bits of the random word, the result space contains four equally likely outcomes. Three re-rolls reach 87.5% probability of a preferred result; five reach 96.9%. Combined with the public currentRandomWord exposure, the owner sees each result before

committing and selects between resolveExternal, emergencyResolve (1× refund), or VRF re-roll, every branch favors the pool wallet over players.

Remediation

It is recommended to replace every revert in the VRF callback path with a silent return, exactly as Chainlink's documentation prescribes, and to additionally block emergencyResolve() once a VRF request is in flight so the re-roll race becomes impossible. VRF request is in flight so the re-roll race becomes impossible.

Additionally, change currentRandomWord from public to internal and emit only keccak256(abi.encode(randomWord)) in ResolvedInternal if off-chain auditability is required, this removes the front-run signal that lets the owner decide which branch to take before committing.

Retest

This issue has been fixed by implementing as per remediation.

Bug ID #L001 [Fixed]

Owner can selectively void unfavorable rounds after observing the committed VRF random word

Vulnerability Type
Business Logic (SC03-LogicErrors)

Severity
Low

Description
After the owner calls closeAndRequestRandomness, drawingStartedAt is set to the close-time block timestamp and a VRF request is dispatched. When Chainlink fulfills the request, fulfillRandomWords writes the random word to the public currentRandomWord storage slot, sets awaitingCrossCheck = true, and emits ResolvedInternal(currentRoundId, randomWord) making the random value visible to anyone watching chain state, including the owner. The contract's outcome derivation in resolveExternal is purely deterministic on this stored word, so the owner can replicate the side, legs, and ODD/EVEN result off-chain with no contract interaction.

At this point the owner faces a free choice between two onlyOwner functions. Calling resolveExternal finalizes the round, paying the winning side at the configured multiplier and clearing the losing side. Calling emergencyResolve instead voids the round, refunding every bettor at 1x into winnings and discarding the random word, and is gated only by block.timestamp >= drawingStartedAt + EMERGENCY_RESOLVE_DELAY (2 minutes from the close, not from the fulfillment). Because typical VRF fulfillment latency on Ethereum mainnet exceeds two minutes, the emergency window is already open at the moment the owner first sees the random word, meaning the owner can compute the round's payout and, if the protocol would lose money on this outcome (for example, the heavily-staked side won), call emergencyResolve to cancel rather than resolveExternal to honor.

Affected Code

- SpaceLadderBettingV7.sol#L301-L310

Impacts
The protocol's fairness invariant is broken: every round becomes a one-sided option held by the owner where favorable outcomes pay out and unfavorable outcomes are voided at 1× refund, structurally guaranteeing the house cannot lose money on any round it cares to monitor.

Remediation
It is suggested to document this risk clearly if the owner is trusted, otherwise make sure that the owner cannot selectively void unfavorable rounds.

Retest
This issue has been fixed by documenting that the owner is totally trusted.

Bug ID #L002 [Fixed]

Sybil Attacker Can Brick Resolution By Inflating Player Arrays

Vulnerability Type
Denial of Service (SCWE-087)

Severity
Low

Description
placeBet() pushes msg.sender into oddPlayers or evenPlayers whenever the caller's prior side stake is zero, with no cap on per-cycle player count. The resolution paths _creditAndClear(), _clearSide(), and _refundAndClear() each loop over the full array in a single transaction to credit winnings, zero stakes, or refund balances.

Because the contract enforces only a minBet floor and accepts any caller, an attacker can fund N fresh addresses and have each place the minimum bet on the same side. Once the array grows large enough that iterating it exceeds the block gas limit, every resolution path reverts on out-of-gas.

Both resolveExternal() and emergencyResolve() depend on these loops, so neither can complete. The round's isDrawing flag stays true permanently, no further bets can be opened, and the contract is bricked.

Affected Code

- SpaceLadderBettingV7.sol#L198-L227
- SpaceLadderBettingV7.sol#L365-L391

Impacts
A Sybil attacker funds enough fresh addresses with minBet stakes to push the combined oddPlayers and evenPlayers iteration cost past the block gas limit. Once the round is closed, every resolution path, normal VRF resolution and the emergency refund fallback, runs out of gas and reverts. The contract enters a permanent stuck state: isDrawing cannot flip back to false, no new round can open, and no honest bettor can recover their stake. The cost to the attacker is bounded (N x minBet plus gas for N transactions), while the damage to the protocol is total and irreversible without an upgrade.

Remediation

It is recommended to bound per-cycle player count at placeBet() so the resolution loops always fit in a single block. The cap should be sized against the worst-case gas cost of the heaviest resolution path with safety margin.

Retest

This issue has been fixed by implementing MAX_PLAYERS_PER_ROUND to 500.

Bug ID #I001 [Fixed]

Owner and pool-wallet operator hold unilateral authority over user funds and protocol liveness

Vulnerability Type
Centralization Risk

Severity
Informational

Description
User stakes are pulled directly into an externally-held poolWallet rather than into the contract itself, so the contract never custodies funds and depends on the pool wallet maintaining both sufficient ERC-20 balance and a sufficient transferFrom allowance for withdrawWinnings to succeed; the pool wallet operator can at any moment revoke approval, drain the wallet, or block the contract from pulling, leaving credited winnings[] entries unredeemable on a first-come-first-served basis with no on-chain recourse for users. Round liveness is fully owner-driven, closeAndRequestRandomness, resolveExternal, and emergencyResolve are all onlyOwner and there is no on-chain time logic, no permissionless finalize path, and no permissionless user-refund function so an absent or uncooperative owner can leave bets stranded indefinitely after placeBet has already moved the user's tokens to the pool wallet.

Affected Code

- SpaceLadderBettingV7.sol#L226

Impacts
A malicious or compromised owner can steal the value of all in-flight bets indirectly by stalling resolution, voiding rounds via emergencyResolve to prevent unfavorable payouts, swapping the VRF coordinator to bias future outcomes, or moving the pool wallet to an address they fully control.

Remediation
It is suggested to document this centralization risk explicitly in the protocol's user-facing documentation.

Retest
This issue has been fixed by documenting that the owner is totally trusted.

Bug ID #I002 [Fixed]

Fee-on-transfer and rebasing tokens are silently unsupported, causing accounting drift and pool-wallet over payouts

Vulnerability Type
Token-compatibility

Severity
Informational

Description
The contract treats the amount argument passed to placeBet as the canonical stake value and immediately writes it to per-side accounting (oddStakeOf, evenStakeOf, oddStaked, evenStaked) before invoking token.safeTransferFrom. This pattern assumes a strict 1:1 correspondence between the requested transfer amount and the recipient's balance increase, which holds only for standard ERC-20 tokens that do not levy a transfer fee, do not rebase balances, and do not adjust transferred amounts.

When the round resolves, _creditAndClear multiplies the recorded stake by multiplierBps and credits winnings[player] from that inflated base, with no relationship to what poolWallet actually holds. The withdrawal path then asks poolWallet to settle the full credited amount via safeTransferFrom, which itself may incur another transfer fee, the user receives less than credited and poolWallet pays more than it received from the original stake. Over many rounds the cumulative drift is fee_rate × total_volume × (multiplier + 1), and because withdrawWinnings settles first-come-first-served, later winners can encounter InsufficientPool reverts even though the on-chain winnings[ ] ledger says they are owed funds. Rebasing tokens introduce a different but related break: stakes recorded mid-round become incorrect when the next rebase event changes balances, desynchronizing oddStaked + evenStaked from poolWallet's actual holdings.

Affected Code

- SpaceLadderBettingV7.sol#L208-L226

Impacts
For any FoT or deflationary token, every round produces a structural shortfall in poolWallet equal to fee_rate × total_volume, compounded by the multiplier on the winning side, so the pool-wallet operator is forced to top up the wallet from external funds to keep withdrawWinnings solvent or

accept that later winners will hit InsufficientPool and lose their entitlement on a first-come-first-served basis.

Remediation
Explicitly document that only standard non-FoT, non-rebasing ERC-20 tokens are supported. If FoT support is required, switch the accounting to balance-delta based, snapshot token.balanceOf(poolWallet) before and after safeTransferFrom in placeBet and use the measured delta as the recorded stake, applying the same pattern symmetrically in withdrawWinnings.

Retest
This issue is fixed by documenting that FoT tokens are not supported.

6. The Disclosure ----------

The Reports provided by CredShields are not an endorsement or condemnation of any specific project or team and do not guarantee the security of any specific project. The contents of this report are not intended to be used to make decisions about buying or selling tokens, products, services, or any other assets and should not be interpreted as such.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. CredShields does not provide any warranty or representation about the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business. The report is not intended to be used as investment advice and should not be relied upon as such.

CredShields Audit team is not responsible for any decisions or actions taken by any third party based on the report.

Your Secure Future Starts Here

CRED SHIELDS

At CredShields, we're more than just auditors. We're your strategic partner in ensuring a secure Web3 future. Our commitment to your success extends beyond the report, offering ongoing support and guidance to protect your digital assets

AICPA SOC
aicpa.org/soc4so

SCS Advocates
OWASP
OWASP Smart Contract Top 10
Security Risks & Vulnerabilities

20A TANJONG PAGAR ROAD, SINGAPORE (088443)

info@credshields.com